

Introduction to Artificial Intelligence

Prof. Marc Toussaint | University of Stuttgart (Machine Learning & Robotics Lab)

1. Foundations of AI, Agent Architectures & Interactive Decision Domains

Artificial Intelligence (AI) is the formal study of computational systems that interact with an environment and make optimal decisions based on available information. Rather than treating intelligence as a collection of biological heuristics, modern AI casts rational behavior as optimization under objective functions and probabilistic inference over latent states.

1.1 The Optimization Paradigm of Intelligence

- **Intelligence as Objective Optimization:** Optimality principles serve as the core scientific language to describe rational behavior. Any behavioral system can be cast as optimal under a specific objective function (analogous to Feynman's principle of least action in physics).
- **The Three Core Problem Classes in AI:** All computational AI challenges can be partitioned into: (1) Continuous and Discrete Numerical Optimization, (2) Categorical and Logical Inference (such as Constraint Satisfaction, Boolean Satisfiability), and (3) Probabilistic Inference over uncertain, partially observed latent variables.
- **Universal Behavioral Objectives:** Generic objectives include maximizing cumulative reward, minimizing predictive entropy (active learning and curiosity), and mastering all controllable degrees of freedom (DOFs) in an environment.

1.2 Formal Modeling of Interactive Domains (POMDPs & Dec-POMDPs)

- **Interaction Cycle:** At each discrete time step t , the world resides in a state s_t in S . The agent perceives an observation y_t in O and receives a scalar reward r_t . The agent executes an action a_t in A , driving the environment into a successor state s_{t+1} drawn from $P(s_{t+1}|s_t, a_t)$.
- **The Markovian State Definition:** A state s_t is a complete and sufficient description of a domain if and only if future observations y_{t+} (where $t+ > t$) are conditionally independent of all historical observations y_{t-} (where $t- < t$) given s_t and intervening actions $a_{t:t+}$, formalized as $I(y_{t+}, y_{t-} | s_t, a_{t:t+})$.
- **Agent Formalization Models:** Agent architectures are categorized by their memory capacity:
 - **Memoryless Reflex Agent:** Direct stimulus-response mapping: y_t maps to a_t (suboptimal in partially observable settings).
 - **Historical Window Agent:** Considers fixed horizon history: $(y_{t-k:t}, a_{t-k:t-1})$ maps to a_t .
 - **Finite State Machine Agent:** Maintains internal state n_t : updates (n_{t-1}, y_t) to n_t and acts n_t to a_t .
 - **Bayesian Belief Agent:** Maintains a full posterior probability distribution $b_t(s_t) = P(s_t|h_t)$ over the underlying world state, updating via Bayes filter (b_{t-1}, a_{t-1}, y_t) to b_t and acting b_t to a_t .
- **Decentralized Multi-Agent Systems (Dec-POMDPs):** Extends interactive domains to n distributed agents with joint action spaces and private observation channels, characterized by NEXP-complete computational complexity.

2. Classical Deterministic Search Algorithms & Heuristic Optimization

Search algorithms address deterministic, fully observable sequential decision problems by traversing a state space graph from an initial state s_0 toward a goal condition S_{goal} while minimizing cumulative step costs.

2.1 Search Problem Formulation & Data Structures

- **Problem Quadruple:** Formally defined by the tuple: S (state space), s_0 (initial state), $Succ(s, a)$ (successor function), S_goal (goal condition), and $Cost(s, a, s')$ (step cost function).
- **States vs. Search Nodes:** A state represents a physical world configuration. A search node n is an algorithmic data structure in memory encapsulating: $State(n)$, $Parent(n)$, $Action(n)$, Path Cost $g(n)$, and Depth $d(n)$.
- **Tree Search vs. Graph Search:** Tree search evaluates paths independently, risking exponential explosion in cyclical graphs. Graph search maintains a Closed Set (explored table); nodes with already-visited states are discarded from the fringe, preventing redundant expansion.

2.2 Uninformed Search Strategies Complexity Analysis

Algorithm	Fringe Policy	Time Comp.	Space Comp.	Optimality
Breadth-First (BFS)	FIFO Queue	$O(b^{(d+1)})$	$O(b^{(d+1)})$ (Exp)	Yes (unit cost)
Uniform-Cost (UCS)	Priority Queue $g(n)$	$O(b^{(C^*/\epsilon)})$	$O(b^{(C^*/\epsilon)})$	Yes (step cost $\geq \epsilon$)
Depth-First (DFS)	LIFO Stack	$O(b^m)$	$O(bm)$ (Linear)	No
Iterative Deepening (IDS)	Iterative DLS	$O(b^d)$	$O(bd)$ (Linear)	Yes (unit cost)

2.3 Informed A* Search & Theoretical Optimality Proofs

- **Evaluation Function Formulation:** $f(n) = g(n) + h(n)$, combining past path cost $g(n)$ with heuristic optimistic cost-to-go estimate $h(n)$.
- **Admissibility Property:** A heuristic $h(n)$ is admissible if $0 \leq h(n) \leq h^*(n)$ for all nodes n , where $h^*(n)$ is the true optimal cost from n to a goal.
- **Consistency (Monotonicity) Property:** A heuristic is consistent if $h(n) \leq c(n, a, n') + h(n')$ for every transition n to n' . Consistency guarantees that $f(n)$ is non-decreasing along any path, eliminating the need to re-open closed nodes in graph search.
- **Proof of A* Tree Search Optimality:** Let G_2 be a suboptimal goal in the fringe ($g(G_2) > C^*$), and let n be an unexpanded node on the optimal path to goal G . Since $h(G_2) = 0$, $f(G_2) = g(G_2) > C^* = g(G)$. Since h is admissible, $f(n) = g(n) + h(n) \leq C^*$. Therefore $f(n) < f(G_2)$, ensuring that A* expands all optimal path nodes before selecting any suboptimal goal.
- **Heuristic Dominance & Relaxation:** If $h_2(n) \geq h_1(n)$ for all n , h_2 dominates h_1 and expands fewer or equal nodes. Admissible heuristics are systematically derived by computing exact solution costs of relaxed problem formulations.
- **Memory-Bounded Search (SMA*):** When memory limit is reached, SMA* drops the worst node (highest f) and backs up its value to its parent, preserving completeness and optimality within available RAM.

3. Probabilistic Information Calculus & Continuous Distributions

Probability theory provides a rigorous mathematical framework for representing and manipulating uncertainty in AI systems, treating probability distributions as quantified degrees of belief.

3.1 Axiomatic Foundations & Inference Calculus

- **Kolmogorov Axioms:** Non-negativity $P(A) \geq 0$, Additivity for disjoint events $P(A \cup B) = P(A) + P(B)$ if $A \cap B$ is empty, and Normalization $P(\Omega) = 1$.
- **Joint, Marginal & Conditional Calculus:** Joint distribution $P(X, Y)$, Marginalization $P(X) = \sum_Y P(X, Y)$, and Conditional probability $P(X|Y) = P(X, Y) / P(Y)$.

- **Bayes' Theorem:** The universal foundation for Bayesian belief updating: $P(X|Y) = P(Y|X)P(X) / P(Y) = (\text{Likelihood} * \text{Prior}) / \text{Normalization}$.
- **Conditional Independence:** X and Y are conditionally independent given Z ($I(X, Y | Z)$) iff $P(X, Y | Z) = P(X|Z)P(Y|Z)$, reducing exponential table parameterizations.

3.2 Discrete and Continuous Probability Distributions

- **Bernoulli & Binomial:** $\text{Bern}(x|\mu) = \mu^x (1-\mu)^{(1-x)}$ for x in $\{0, 1\}$. Binomial models counts over n trials: $\text{Bin}(m|n, \mu) = \binom{n}{m} \mu^m (1-\mu)^{(n-m)}$.
- **Beta Distribution:** Conjugate prior for Bernoulli parameter μ in $[0, 1]$: $\text{Beta}(\mu|a, b)$ proportional to $\mu^{a-1} (1-\mu)^{b-1}$. Posterior update given a_D successes and b_D failures is simply $\text{Beta}(\mu | a + a_D, b + b_D)$.
- **Multinomial & Dirichlet:** Dirichlet distribution parameterizes the K-simplex: $\text{Dir}(\mu|\alpha)$ proportional to $\prod_k \mu_k^{(\alpha_k - 1)}$, acting as the conjugate prior to the Multinomial distribution with update $\alpha + m$.
- **Multivariate Gaussian Distribution:** Parameterized in standard covariance form $N(x|\mu, \Sigma)$ and canonical information form $N[x|a, A]$ with precision matrix $A = \Sigma^{-1}$ and potential vector $a = \Sigma^{-1} \mu$.

3.3 Information Theory & Sampling Methods

- **Entropy:** Expected information content or uncertainty: $H(p) = E_p[-\log p(x)] = - \int p(x) \log p(x) dx$.
- **Kullback-Leibler (KL) Divergence:** Relative entropy measuring distribution mismatch: $D_{KL}(p || q) = \int p(x) \log (p(x) / q(x)) dx \geq 0$ (Gibbs' inequality).
- **Rejection Sampling:** Draws candidate x from proposal $q(x)$ satisfying $p(x) \leq M q(x)$; accepts with probability $p(x) / (M q(x))$ to yield exact i.i.d. samples.
- **Importance Sampling:** Estimates expectations $E_p[f(x)]$ by drawing x_i from $q(x)$ and weighting by likelihood ratio $w_i = p(x_i) / q(x_i)$.

4. Multi-Armed Bandits, Monte Carlo Tree Search & Adversarial Games

Decision-making under uncertainty involves balancing exploration (gathering information to improve future choices) with exploitation (selecting the best currently known action).

4.1 Multi-Armed Bandits & The UCB1 Algorithm

- **Bandit Problem Formulation:** Given n payoff distributions $P(y|\theta_i)$, find an action policy mapping history to action maximizing cumulative expected return.
- **UCB1 Upper Confidence Bound:** Derived from Hoeffding's inequality, selects machine i that maximizes: $a_t = \arg\max_i [\hat{y}_i + \beta * \sqrt{2 * \ln(n) / n_i}]$.
- **Optimism in the Face of Uncertainty:** The exploration term proportional to $\sqrt{\ln(n) / n_i}$ bounds total cumulative regret logarithmically to $O(\ln T)$.

4.2 Monte Carlo Tree Search (MCTS) & UCT

- **The 4-Step MCTS Iteration Cycle:** Repeats until computational budget expires:
 - **1. Selection:** Traverse existing search tree from root using Tree Policy (UCB) until an unexpanded node is reached.
 - **2. Expansion:** Create one or more child nodes according to available domain actions.
 - **3. Simulation (Rollout):** Execute a fast randomized default rollout policy to terminal state, obtaining sample return Δ .

- **4. Backpropagation:** Propagate reward Delta upward to all ancestor nodes: $n(v) \leftarrow n(v) + 1$, $Q(v) \leftarrow Q(v) + \Delta$.
- **Upper Confidence bounds applied to Trees (UCT):** Applies UCB at every tree branching node: $\text{argmax}_{v'} [Q(v')/n(v') + 2 C_p * \sqrt{2 * \ln(n(v)) / n(v')}]$.
- **MCTS in POMDPs (POMCP):** Silver & Veness framework: Search tree nodes represent action-observation histories $h(v)$ paired with particle sets $S(v)$ representing unobserved world states.

4.3 Game Playing: Minimax & Alpha-Beta Pruning

- **Minimax Principle:** For 2-player deterministic zero-sum perfect information games, computes optimal move against an optimal opponent by recursively maximizing and minimizing position utilities.
- **Alpha-Beta Pruning Algorithm:** Branch-and-bound optimization maintaining alpha (highest value guaranteed to MAX) and beta (lowest value guaranteed to MIN). A branch is pruned whenever $\alpha \geq \beta$.
- **Computational Efficiency:** Perfect move ordering reduces effective branching factor from b^m to $b^{(m/2)}$, doubling searchable game depth.

5. Markov Decision Processes & Dynamic Programming

Markov Decision Processes (MDPs) provide the foundational mathematical framework for sequential decision-making in stochastic environments.

5.1 Mathematical MDP Formalism

- **MDP Components:** Tuple $(S, A, P(s'|s, a), R(s, a), \gamma)$, where discount factor γ in $[0, 1]$ ensures convergence over infinite horizons.
- **Value Function $V^\pi(s)$:** Expected discounted return under policy π : $V^\pi(s) = E_\pi [\sum_t \gamma^t r_t \mid s_0 = s]$.
- **Action-Value Function $Q^\pi(s, a)$:** Expected discounted return taking action a in state s : $Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s')$.

5.2 Bellman Equations & Dynamic Programming Algorithms

- **Bellman Optimality Equations:** Fundamental fixed-point recursive definitions:
 - **State-Value Optimality:** $V^*(s) = \max_a [R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s')]$.
 - **Action-Value Optimality:** $Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a')$.
- **Value Iteration Algorithm:** Successive approximation updating $V_{k+1}(s) \leftarrow \max_a [R(s, a) + \gamma \sum_{s'} P(s'|s, a) V_k(s')]$.
- **Proof of Convergence (Contraction Mapping):** Let $\Delta_k = \|Q^* - Q_k\|_\infty = \max_{\{s, a\}} |Q^*(s, a) - Q_k(s, a)|$. Applying the Bellman operator yields $\|Q^* - Q_{k+1}\|_\infty \leq \gamma * \Delta_k$. By Banach Fixed-Point Theorem (since $\gamma < 1$), Q_k converges to Q^* exponentially with rate γ .
- **Policy Iteration:** Alternates between (1) Policy Evaluation (solving linear system $V^\pi = (I - \gamma P^\pi)^{-1} R^\pi$) and (2) Policy Improvement ($\pi_{k+1}(s) \leftarrow \text{argmax}_a Q^\pi(s, a)$).

5.3 Dynamic Programming in Belief Space (Belief MDPs)

- **Belief MDP Concept:** When the environment model θ or state s_t is unknown, the agent defines a continuous MDP over its belief state $b_t(\theta) = P(\theta | h_t)$.
- **Belief State Value Function:** Formulates optimal trade-off between knowledge acquisition and reward: $V_t^*(b) = \max_a [R(a, b) + \int P(y|b, a) V_{t+1}^*(b'_{\text{next}}) dy]$.

- **Automatic Exploration-Exploitation:** Belief space DP automatically plans optimal exploration actions without requiring heuristic exploration parameters.

6. Reinforcement Learning: Model-Free, Model-Based & Policy Search

Reinforcement Learning (RL) addresses the problem of an agent learning optimal behavioral policies in an unknown environment through trial-and-error interactions.

6.1 Model-Free Temporal Difference (TD) Learning

- **Q-Learning (Off-Policy TD Control):** Learns $Q^*(s, a)$ directly without learning transition dynamics: $Q(s, a) \leftarrow Q(s, a) + \alpha * [r + \gamma * \max_{a'} Q(s', a') - Q(s, a)]$.
- **SARSA (On-Policy TD Control):** Updates action values based on actual next action chosen under policy: $Q(s, a) \leftarrow Q(s, a) + \alpha * [r + \gamma * Q(s', a) - Q(s, a)]$.
- **Eligibility Traces (TD(λ) / Q(λ)):** Multi-step credit assignment maintaining trace $e(s, a) \leftarrow \gamma * \lambda * e(s, a) + 1$, propagating TD error across all recently visited states.
- **Experience Replay Buffer:** Stores historical transitions $D = \{(s_i, a_i, r_i, s_{i+1})\}$, breaking temporal correlations and stabilizing deep neural network Q-learning.

6.2 Exploration Strategies & PAC-MDP Theory

- **Epsilon-Greedy & Optimistic Initialization:** Balances random actions with probability epsilon and greedy exploitation with 1-epsilon.
- **R-MAX Algorithm (Brafman & Tennenholtz):** Model-based PAC-MDP algorithm. Transitions with visit counts $c(s, a) < m$ are assigned maximal reward $R_{\max}$ and self-loops, provably converging in polynomial sample complexity.
- **Bayesian Exploration Bonus (BEB):** Adds parametric uncertainty bonus $R^*(s, a) = R_{\hat{}}(s, a) + \beta / (1 + \alpha c(s, a))$ to incentivize unvisited transitions.

6.3 Policy Search, Imitation Learning & Inverse RL

- **REINFORCE Policy Gradient Theorem:** Directly parameterizes stochastic policy $\pi_{\theta}(a|s)$: $\text{grad}_{\theta} J(\theta) = E [\sum_t \text{grad}_{\theta} \log \pi_{\theta}(a_t|s_t) Q^{\pi}(s_t, a_t)]$.
- **Behavioral Cloning & Imitation Learning:** Directly trains supervised classification/regression models from expert state-action trajectories $D = \{(s_t, a_t)\}$.
- **Apprenticeship Learning via Inverse RL:** Recovers underlying latent reward function $R(s, a) = w^T \phi(s, a)$ that renders demonstrated expert behavior optimal via max-margin optimization.

7. Constraint Satisfaction Problems (CSPs) & Arc Consistency

A Constraint Satisfaction Problem (CSP) consists of finding state assignments for a set of variables that strictly satisfy a collection of categorical constraints.

7.1 Formal Definition & Constraint Graphs

- **CSP Triplet (X, D, C):** Variables $X = \{X_1, \dots, X_n\}$, finite domains $D = \{D_1, \dots, D_n\}$, and constraints $C = \{C_1, \dots, C_K\}$ where $C_k = (I_k, c_k)$ restricts variable subsets X_{I_k} .
- **Constraint Graph Structure:** Bipartite graph with variable nodes and constraint nodes; edges denote variable scope membership.

7.2 Backtracking Search & Heuristics

- **Commutativity & Single-Variable Assignment:** Because variable assignment order is commutative, DFS fixes a single variable per tree level, reducing branching factor from $b = (n-1)d$ to $b = d$ (yielding d^n leaves).
- **Minimum Remaining Values (MRV):** Selects variable with the fewest legal domain values (fail-first principle).
- **Degree Heuristic:** Selects variable involved in the largest number of constraints on remaining unassigned variables (tie-breaker for MRV).
- **Least Constraining Value (LCV):** Orders values to leave the maximum flexibility for neighboring variables (fail-last principle).

7.3 Constraint Propagation & The AC-3 Algorithm

- **Arc Consistency Definition:** Directed arc X_i to X_j is consistent iff for every value x in D_i , there exists an allowed y in D_j satisfying constraint $C(X_i, X_j)$.
- **AC-3 Algorithm Mechanics:** Initializes queue with all directed arcs; when domain D_i loses a value, re-inserts all neighbor arcs (X_k, X_i) to propagate domain reductions in $O(n^2 d^3)$ time.
- **Tree-Structured CSPs:** CSPs without loops are solved in linear time $O(n d^2)$ by: (1) Topological ordering, (2) Backward constraint propagation from leaves to root, and (3) Forward backtrack-free assignment.
- **Cutset Conditioning:** Instantiates a cutset of c variables such that remaining graph is a tree, reducing solve time to $O(d^c * (n-c)d^2)$.

8. Probabilistic Graphical Models & Exact/Approximate Inference

Graphical models combine probability theory with graph data structures to represent structured joint distributions over high-dimensional spaces.

8.1 Bayesian Networks & D-Separation

- **Directed Acyclic Graph (DAG) Factorization:** $P(X_{1:n}) = \prod_{i=1}^n P(X_i \mid \text{Parents}(X_i))$.
- **D-Separation Rules:** Path between X and Y is blocked by conditioning set Z iff: (1) Head-to-tail node in path in Z , (2) Tail-to-tail node in Z , or (3) Head-to-head (collider) node AND its descendants are not in Z .
- **Explaining Away Effect:** In collider structures $X \rightarrow Z \leftarrow Y$, observing Z creates conditional dependence between marginally independent causes X and Y .

8.2 Factor Graphs & Message Passing Algorithms

- **Factor Graph Representation:** $P(X_{1:n}) = (1/Z) * \prod_{k=1}^K f_k(X_{\partial k})$, where factors f_k are non-negative potential functions.
- **Variable Elimination Algorithm:** Computes marginals by pushing summation inward across factor products, generating intermediate message factors $\mu_{k \rightarrow i}(X_i)$ via dynamic programming.
- **Belief Propagation (Sum-Product on Trees):** Computes exact marginals via recursive message equations: Factor-to-Variable message $\mu_{k \rightarrow i}(X_i) = \sum_{X_{\partial k - \{i\}}} f_k(X_{\partial k}) \prod_{j \neq i} \mu_{j \rightarrow k}(X_j)$; Variable-to-Factor message $\mu_{i \rightarrow k}(X_i) = \prod_{k' \neq k} \mu_{k' \rightarrow i}(X_i)$.
- **Loopy Belief Propagation:** Iterates BP equations on loopy graphs; fixed points correspond to stationary states of the Bethe Free Energy approximation.
- **Junction Tree Algorithm:** Converts loopy graphs into exact trees via (1) Moralization, (2) Triangulation, and (3) Clique tree message passing.

- **MAP Inference (Max-Product Algorithm):** Replaces sum with max in message passing equations to compute most probable global configuration $X_{\text{MAP}} = \operatorname{argmax}_X P(X)$.

9. Dynamic Models: Hidden Markov Models & Kalman Filtering

Dynamic models describe time-series processes where latent state variables evolve over discrete or continuous time under observation noise.

9.1 Hidden Markov Models (HMMs)

- **HMM Factorization:** $P(X_{0:T}, Y_{0:T}) = P(X_0) * \prod_{t=1}^T P(X_t | X_{t-1}) * \prod_{t=0}^T P(Y_t | X_t)$.
- **Core HMM Inference Tasks:** Filtering $P(X_t | Y_{0:t})$, Smoothing $P(X_t | Y_{0:T})$, Prediction $P(X_{t+k} | Y_{0:t})$, and Likelihood $P(Y_{0:T})$.
- **Forward-Backward Matrix Algorithm:** Forward message $\alpha_t = P^T(\alpha_{t-1} \circ \rho_{t-1})$ and Backward message $\beta_t = P(\beta_{t+1} \circ \rho_{t+1})$, yielding exact smoothed posterior $P(X_t | Y_{0:T})$ proportional to $\alpha_t \circ \rho_t \circ \beta_t$.
- **Viterbi Algorithm:** Computes the most likely sequence of hidden states $\operatorname{argmax}_{X_{0:T}} P(X_{0:T} | Y_{0:T})$ using max-product message passing in linear time $O(T |S|^2)$.

9.2 Continuous Linear Gaussian Systems: The Kalman Filter

- **State Space Formulation:** Linear Gaussian transitions $x_{t+1} = A x_t + B u_t + \epsilon_{t,x}$ (where $\epsilon_{t,x} \sim N(0, Q)$) and observations $y_t = C x_t + \epsilon_{t,y}$ (where $\epsilon_{t,y} \sim N(0, W)$).
- **Kalman Prediction Step:** $\hat{x}_{t|t-1} = A \hat{x}_{t-1|t-1}$, $\Sigma_{t|t-1} = A \Sigma_{t-1|t-1} A^T + Q$.
- **Kalman Measurement Update:** Kalman Gain $K_t = \Sigma_{t|t-1} C^T (C \Sigma_{t|t-1} C^T + W)^{-1}$; State Update $\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t (y_t - C \hat{x}_{t|t-1})$; Covariance Update $\Sigma_{t|t} = (I - K_t C) \Sigma_{t|t-1}$.

10. Deep Learning, Differentiable Graphs & Modern Optimization

Neural networks are parameterized differentiable computation graphs trained via gradient descent to approximate complex nonlinear mappings.

10.1 Empirical Risk Minimization & Backpropagation

- **Empirical Loss Minimization:** Given dataset $D = \{(x_i, y_i)\}$, minimize empirical risk $L(\theta) = \sum_i \text{loss}(f_\theta(x_i), y_i) + \lambda * \Omega(\theta)$.
- **Analytical Vectorized Backpropagation:** Recursive application of chain rule across layer activations $x^l = \sigma(z^l)$ and pre-activations $z^l = W^{l-1} x^{l-1}$; Layer error propagation $\delta^l = [W^l]^T \delta^{l+1} \circ \sigma'(z^l)$; Weight gradient $dL/dW^l = \delta^l [x^{l-1}]^T$.
- **Vanishing Gradients & Modern Activations:** Sigmoid derivatives $\sigma'(z) \leq 0.25$ cause exponential gradient decay in deep architectures; Rectified Linear Units (ReLU: $\sigma(z) = \max(0, z)$) preserve gradient flow across deep topologies.

10.2 Structural Priors: CNNs, LSTMs & Stochastic Optimizers

- **Convolutional Neural Networks (CNNs):** Imposes local receptive fields, shared kernel parameters ($k \times k$), and spatial translation invariance, reducing parameters compared to dense matrices.

- **Long Short-Term Memory (LSTM):** Preserves long-term temporal error gradients via linear memory cell carousel $c_t = \Gamma_f \odot c_{t-1} + \Gamma_u \odot \tilde{c}_t$, gated by forget Γ_f , update Γ_u , and output Γ_o sigmoid gates.
- **Adam Optimization Algorithm:** Combines running first moment (momentum) $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$ and second raw moment $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ with bias corrections to update parameters $\theta_t \leftarrow \theta_{t-1} - (\alpha / (\sqrt{\hat{v}_t} + \epsilon)) * \hat{m}_t$.

11. Explainable AI (XAI) & Sensitivity Analysis Frameworks

Explainable AI (XAI) develops mathematical mechanisms to explain the decisions of nonlinear black-box models by attributing outputs to human-interpretable input features.

11.1 Sensitivity Analysis in Constrained Optimization

- **KKT Optimality Sensitivity:** For optimal solution $x^* = \operatorname{argmin}_x f(x)$ subject to $g(x) \leq 0$, $h(x) = 0$, evaluates infinitesimal variations of the objective and constraints by solving the primal-dual Newton sensitivity block matrix equation.
- **Differentiable Optimization:** Enables optimization problem layers to be embedded directly into end-to-end autodiff computation graphs (such as TensorFlow or PyTorch).

11.2 Feature Attribution in Deep Neural Networks

- **Guided Backpropagation:** Combines standard backpropagation with deconvolution, zeroing negative gradients and negative activations: $R_i^l = [x_i^l > 0] [R_i^{l+1} > 0] R_i^{l+1}$ to isolate sharp visual features.
- **Layer-Wise Relevance Propagation (LRP) & Deep Taylor Decomposition:** Decomposes total network output conservatively into sum of input relevances $f(x) = \sum_i R_i$, exploiting piecewise linearity of ReLU networks to perform exact 1st-order Taylor expansions.
- **Influence Functions (Koh & Liang):** Uses empirical Hessian inverse H^{-1} to compute sensitivity of model parameters to the upweighting or removal of individual training data points without retraining: $I_{\text{up,loss}}(z, z_{\text{test}}) = -\operatorname{grad}_{\theta} \text{loss}(z_{\text{test}}, \theta_{\text{hat}})^T H_{\{\theta_{\text{hat}}\}}^{-1} \operatorname{grad}_{\theta} \text{loss}(z, \theta_{\text{hat}})$.

12. Symbolic Logic, Knowledge Representation & Relational Probabilistic AI

Unifying first-order relational representations with probabilistic modeling enables AI systems to generalize across arbitrary domains, objects, and relational structures.

12.1 Propositional Logic & Resolution Refutation

- **Syntax & Truth Semantics:** Atomic symbols P, Q, R , connectives (negation, conjunction, disjunction, implication, biconditional), and model satisfaction $M(\alpha) \subseteq M(\beta)$ for entailment $\alpha \models \beta$.
- **Conjunctive Normal Form (CNF):** Conjunction of disjunctions of literals. Any propositional formula converts to CNF via: (1) Eliminate biconditionals, (2) Eliminate implications, (3) Apply De Morgan's laws to push negation inward, and (4) Distributive expansion.
- **Resolution Inference Rule:** Sound and refutationally complete proof system: from $(l_1 \text{ or } \dots \text{ or } l_i \text{ or } \dots \text{ or } l_k)$ and $(m_1 \text{ or } \dots \text{ or not } l_i \text{ or } \dots \text{ or } m_n)$, derive resolvent $(l_1 \text{ or } \dots \text{ or } l_{i-1} \text{ or } l_{i+1} \text{ or } \dots \text{ or } m_n)$.
- **Forward & Backward Chaining on Horn Clauses:** Linear-time inference for knowledge bases restricted to definite Horn clauses (conjunction of p_i implies q).

12.2 First-Order Logic (FOL) & Generalized Modus Ponens

- **FOL Expressive Syntax:** Constants (objects), Predicates (relations), Functions, Variables, and Quantifiers (Universal, Existential) satisfying duality $\forall x P(x) \equiv \neg \exists x \neg P(x)$.
- **Syntactic Unification:** Computes Most General Unifier (MGU) substitution $\theta = \text{UNIFY}(\alpha, \beta)$ such that $\alpha\theta = \beta\theta$.
- **Generalized Modus Ponens (GMP):** Derives $q\theta$ from premises $p'_1, \dots, p'_n$ and rule $(p_1 \text{ and } \dots \text{ and } p_n \text{ implies } q)$ whenever θ unifies each (p'_i, p_i) .
- **First-Order Resolution in CNF:** Requires Skolemization to replace existential variables with Skolem functions of enclosing universal variables before resolving clauses.

12.3 Relational MDPs & Statistical Relational AI

- **Planning Domain Definition Language (PDDL / STRIPS):** Describes deterministic relational action schemas: $\text{Action}(a) : \text{PRECOND} \rightarrow \text{EFFECT}$ under the STRIPS frame assumption (unmentioned literals persist).
- **Noisy Deictic Rules (NDRs):** Probabilistic extension of PDDL rules modeling stochastic transition distributions $P(s' | s, a) = \sum_i p_{\{r, i\}} P(s' | \Omega_{\{r, i\}}, s)$, enabling compact, learnable models over exponentially large state spaces.
- **Markov Logic Networks (MLNs):** First-order knowledge base with real-valued weights $\{(F_i, w_i)\}$. For any concrete domain D of objects, instantiates a ground factor graph with probability: $P(X = x) = (1/Z) * \exp(\sum_i w_i n_i(x))$, where $n_i(x)$ is the number of true groundings of formula F_i .
- **The Role of Uncertainty in AI:** Uncertainty acts as mathematical regularization, enabling compression, abstraction, and sample-efficient relational learning from limited data.